# Platforma Edukacji

Dokumentacja Projektowa PZSP2

Wersja 1.0
28.03.2026
Semestr 26L
Zespół nr 3 w składzie:


- Krzysztof Wojtaszko
- Maksymilian Zieliński
- Mateusz Jakubiak
- Sebastian Wojciechowski

Mentor Zespołu: Dr inż. Marcin Szlenk
Właściciel tematu: Dr inż. Krystian Radlak


## Spis treści


1 Wprowadzenie 2
1.1 Cel projektu 2
1.2 Wstępna wizja projektu 2
2 Metodologia wytwarzania 2
3 Analiza wymagań 2
3.1 Wymagania użytkownika i biznesowe 2
3.2 Wymagania funkcjonalne i niefunkcjonalne 2
3.3 Przypadki użycia 2
3.4 Potwierdzenie zgodności wymagań 2
4 Definicja architektury 3
5 Dane trwałe 3
5.1 Model logiczny danych 3
5.2 Przetwarzanie i przechowywanie danych 3
6 Specyfikacja analityczna i projektowa 3
7 Projekt standardu interfejsu użytkownika 3
8 Specyfikacja testów 3
*9 Wirtualizacja/konteneryzacja* 4
10 Bezpieczeństwo 4

11 Podręcznik użytkownika 4
12 Podręcznik administratora 4
13 Podsumowanie 4
14 Bibliografia 4

# Wprowadzenie

## 1.1 Cel projektu

Celem projektu jest stworzenie prototypu platformy internetowej umożliwiającej zarządzanie szkołą edukacyjną oraz obsługę płatności za oferowane kursy. Opracowany system powinien wspierać cztery typy użytkowników:

- **administratorów** – odpowiedzialnych za definiowanie kursów, przypisywaniem nauczycieli do kursów oraz ustawianie harmonogramów kursów przy uproszczonym założeniu, że mamy jedną lub dwie lekcje w tygodniu o stałej porze oraz monitorowanie realizacji miesięcznych płatności,
- **nauczycieli** – posiadających możliwość dodawania materiałów dydaktycznych do prowadzonych kursów, w tym tworzenia lekcji oraz przypisywania do nich materiałów edukacyjnych, a także wpisywania obecności uczniom,
- **rodziców** – mogących przeglądać kalendarz zajęć, sprawdzać obecność uczniów, otrzymywać informacje dotyczące płatności oraz zapoznawać się z informacją zwrotną przekazywaną przez nauczycieli,
- **uczniów** – mających dostęp do harmonogramu nadchodzących zajęć w formie kalendarza oraz listy dotyczasowych lekcji z ich tytułem oraz dostępem do materiałów szkoleniowych udostępnianych poprzez linki do folderów Google Drive przypisanych do konkretnych lekcji.

System powinien umożliwiać logowanie rodzicom oraz uczniom współdzielenie konta przy użyciu adresu e-mail oraz hasła rodzica. Po zalogowaniu użytkownik powinien mieć możliwość wyboru widoku panelu rodzica lub ucznia. Zakłada się, że jeden rodzic może być powiązany z wieloma profilami uczniów.

## 1.2 Wstępna wizja projektu

**Główne założenia działania:**

- administrator zarządza kursami, nauczycielami i harmonogramami,
- nauczyciel dodaje lekcje, materiały i obecności,
- rodzic monitoruje postępy, płatności i harmonogram,
- uczeń ma dostęp do lekcji i materiałów.

**System umożliwia:**

- współdzielenie konta przez rodzica i ucznia,
- dostęp do materiałów poprzez integrację z Google Drive,
- przechowywanie danych użytkowników i kursów w bazie danych

System będzie działał w modelu klient-serwer jako aplikacja webowa.

## Metodologia wytwarzania

Organizacja pracy w projekcie

Projekt realizowany jest zgodnie z podejściem iteracyjnym, zgodnym z metodologią Agile.

Proces wytwarzania obejmuje następujące etapy:

- analiza wymagań,
- projektowanie,
- implementacja,
- testowanie,
- wdrożenie i rozwój.

Zastosowane elementy metodyki:

- iteracyjny rozwój systemu (ciągłe doprecyzowywanie wymagań),
- inkrementalne dostarczanie funkcjonalności,
- regularna walidacja wymagań z interesariuszami.

W ramach inżynierii wymagań stosowane są:

- pozyskiwanie wymagań (elicitation),
- analiza wymagań,
- specyfikacja wymagań,
- walidacja wymagań.

Wymagania zapisywane są:

- w języku naturalnym (np. „System powinien…”),
- w formie historii użytkownika,
- w postaci przypadków użycia.

Organizacja pracy:

- podział ról (np. backend, frontend, dokumentacja),
- wykorzystanie repozytorium kodu,
- komunikacja zespołu przez messenger,
- dokumentacja tworzona przyrostowo zgodnie z wymaganiami.

# Analiza wymagań

## 3.1 Wymaga użytkownika i biznesowe

### Wymagania biznesowe

**Cele biznesowe**

- Usprawnienie organizacji kursów, harmonogramów oraz przypisania nauczycieli.
- Umożliwienie obsługi płatności online za kursy w modelu miesięcznym.
- Zapewnienie jednego miejsca do komunikacji między szkołą, nauczycielem, rodzicem i uczniem.
- Ograniczenie ręcznego prowadzenia obecności i informacji o płatnościach.

**Potrzeby biznesowe**

- Centralne przechowywanie danych o kursach, użytkownikach i harmonogramach.
- Możliwość szybkiego sprawdzenia statusu opłat.
- Łatwy dostęp do materiałów dydaktycznych poprzez Google Drive.
- Prosty panel administracyjny do zarządzania szkołą.

**Problemy do rozwiązania**

- Rozproszone informacje o kursach i materiałach.
- Brak wygodnego dostępu do harmonogramów dla rodziców i uczniów.
- Ręczne pilnowanie płatności i obecności.
- Trudność w przekazywaniu informacji zwrotnej rodzicom.

### Wymagania użytkownika

**Administrator**

- Chce dodawać, edytować i usuwać kursy.
- Chce przypisywać nauczycieli do kursów.
- Chce ustawiać stały harmonogram zajęć (1–2 razy w tygodniu).
- Chce monitorować status płatności rodziców.
- Chce komunikować się z rodzicami i innymi pracownikami.

**Nauczyciel**

- Chce tworzyć lekcje w ramach kursu.
- Chce dodawać materiały edukacyjne jako linki do Google Drive.
- Chce wpisywać obecność uczniów.
- Chce zostawiać krótką informację zwrotną dla rodzica.
- Chce komunikować się z rodzicami i innymi pracownikami.

**Rodzic**

- Chce widzieć kalendarz zajęć dziecka.
- Chce sprawdzać obecności.
- Chce otrzymywać informację o opłatach.
- Chce czytać komentarze od nauczyciela.
- Chce mieć dostęp do wielu profili dzieci z jednego konta.
- Chce komunikować się z pracownikami szkoły.

**Uczeń**

- Chce widzieć harmonogram najbliższych zajęć.
- Chce mieć listę odbytych lekcji.
- Chce otwierać materiały przypisane do lekcji.

**Cechy użytkowe**

- Intuicyjny interfejs.
- Czytelny kalendarz zajęć.
- Łatwe przełączanie widoku rodzic/uczeń.
- Prosta nawigacja między kursami i lekcjami.

## Wymagania systemowe

**Wymagania systemowe**

- System webowy dostępny przez przeglądarkę.
- Logowanie e-mail + hasło.
- Wspólne konto rodzica i ucznia z wyborem widoku po zalogowaniu.
- Baza danych przechowująca użytkowników, kursy, lekcje, obecności i płatności.
- Integracja z linkami Google Drive dla materiałów.
- Moduł prostych płatności online (np. status: opłacone / nieopłacone).
- System przesyłania wiadomości między pracownikami i rodzicami.

## 3.2 Wymagania funkcjonalne i niefunkcjonalne

## Wymagania funkcjonalne

**Zarządzanie użytkownikami**

- System powinien umożliwiać rodzicowi i nauczycielowi samodzielną rejestrację konta.
- System powinien umożliwiać administratorowi tworzenie kont użytkowników.
- System powinien umożliwiać wszystkim użytkownikom logowanie przy użyciu adresu e-mail i hasła.
- System powinien umożliwiać rodzicowi powiązanie wielu profili uczniów z jednym kontem.
- System powinien umożliwiać rodzicowi i uczniowi wybór widoku (rodzic/uczeń) po zalogowaniu.
- System powinien umożliwiać wszystkim użytkownikom edycję własnych danych.
- System powinien umożliwiać administratorowi edycję i dezaktywację kont użytkowników.

**Zarządzanie kursami**

- System powinien umożliwiać administratorowi tworzenie kursów.
- System powinien umożliwiać administratorowi edycję i usuwanie kursów.
- System powinien umożliwiać administratorowi przypisywanie nauczycieli do kursów.
- System powinien umożliwiać administratorowi przypisywanie uczniów do kursów.
- System powinien umożliwiać administratorowi, nauczycielowi, rodzicowi i uczniowi przegląd listy kursów (w zakresie przypisanym do użytkownika).

**Komunikacja i wiadomości**

- System powinien umożliwiać wszystkim użytkownikom wysyłanie wiadomości do innych użytkowników.
- System powinien umożliwiać odbieranie i przeglądanie wiadomości przez wszystkich użytkowników.

- System powinien umożliwiać nauczycielowi wysyłanie wiadomości do administratora kursu w sprawie zmian harmonogramu.
- System powinien umożliwiać administratorowi odpowiadanie na wiadomości dotyczące harmonogramu.
- System powinien umożliwiać prowadzenie historii rozmów między użytkownikami.
- System powinien umożliwiać powiadamianie użytkownika o nowych wiadomościach.

**Zarządzanie harmonogramem**

- System powinien umożliwiać administratorowi tworzenie harmonogramu zajęć dla kursu.
- System powinien umożliwiać administratorowi edycję harmonogramu.
- System powinien umożliwiać administratorowi ustawienie stałych terminów zajęć (np. 1–2 razy w tygodniu).
- System powinien umożliwiać wszystkim użytkownikom wyświetlanie kalendarza zajęć w ich zakresie.

**Zarządzanie lekcjami**

- System powinien umożliwiać nauczycielowi odwoływanie lekcji w ramach kursu.
- System powinien umożliwiać uczniowi i rodzicowi przegląd listy odbytych lekcji.

**Materiały edukacyjne**

- System powinien umożliwiać nauczycielowi dodawanie materiałów do lekcji w formie linków.
- System powinien umożliwiać wszystkim użytkownikom będących członkami kursu przegląd materiałów przypisanych do lekcji.
- System powinien umożliwiać nauczycielowi edycję i usuwanie materiałów.

**Obecności**

- System powinien umożliwiać nauczycielowi oznaczanie obecności uczniów.
- System powinien umożliwiać nauczycielowi edycję obecności uczniów.
- System powinien umożliwiać rodzicowi przegląd obecności dziecka.
- System powinien umożliwiać uczniowi przegląd własnej obecności.

**Płatności**

- System powinien umożliwiać administratorowi rejestrowanie płatności za kursy.
- System powinien umożliwiać administratorowi manualną zmianę statusu płatności (opłacone / nieopłacone).
- System powinien umożliwiać rodzicowi przegląd statusu i historii płatności.
- System powinien umożliwiać administratorowi monitorowanie płatności wszystkich użytkowników.

**Komunikacja i informacje**

- System powinien umożliwiać nauczycielowi dodawanie informacji zwrotnej dla rodzica.
- System powinien umożliwiać rodzicowi przegląd komentarzy nauczyciela.
- System powinien umożliwiać administratorowi i nauczycielowi publikowanie informacji o kursie.

- System powinien umożliwiać wszystkim użytkownikom przegląd podstawowych informacji o kursie i lekcjach.

## Wymagania niefunkcjonalne

**Wydajność**

- System powinien obsługiwać wielu użytkowników jednocześnie bez zauważalnych opóźnień.
- Czas odpowiedzi systemu nie powinien przekraczać kilku sekund dla standardowych operacji.

**Dostępność**

- System powinien być dostępny 24/7 z wyjątkiem zaplanowanych przerw serwisowych.

**Bezpieczeństwo**

- System powinien wymagać uwierzytelnienia użytkownika.
- System powinien zapewniać autoryzację dostępu (różne role użytkowników).
- System powinien chronić dane użytkowników przed nieautoryzowanym dostępem.

**Użyteczność**

- Interfejs powinien być intuicyjny i czytelny.
- System powinien umożliwiać łatwą nawigację między funkcjami.
- System powinien być dostosowany do różnych poziomów umiejętności użytkowników.

**Niezawodność**

- System powinien minimalizować ryzyko utraty danych.
- System powinien umożliwiać odzyskanie danych po awarii.

**Skalowalność**

- System powinien umożliwiać obsługę rosnącej liczby użytkowników i kursów.

**Zgodność**

- System powinien spełniać wymagania dotyczące ochrony danych osobowych.

**Przenośność**

- System powinien działać w przeglądarce internetowej na różnych urządzeniach (komputer, tablet, smartfon).

## Wymagania procesowe

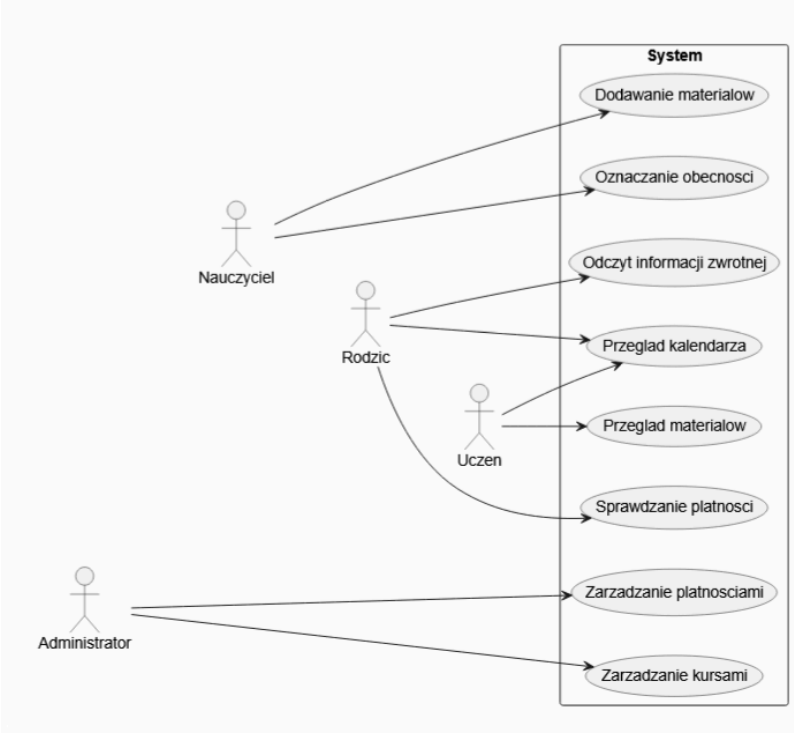
**Przechowywanie wymagań**

- Wymagania powinny być utrzymywane w repozytorium projektu i powiązane z jego architekturą i testami, co można osiągnąć np. za pomocą zastosowania biblioteki Sphinx Needs.

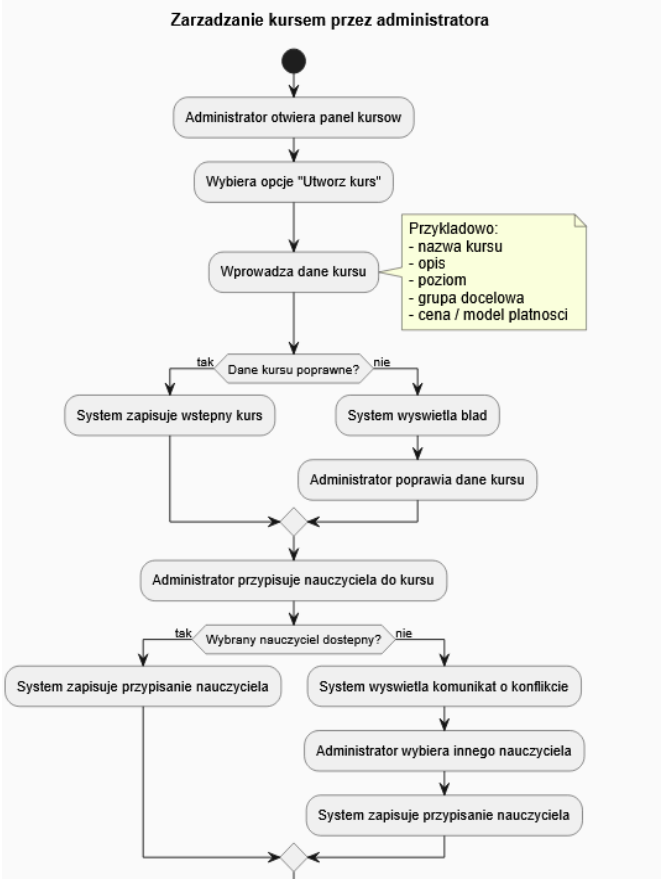
## 3.3 Przypadki użycia

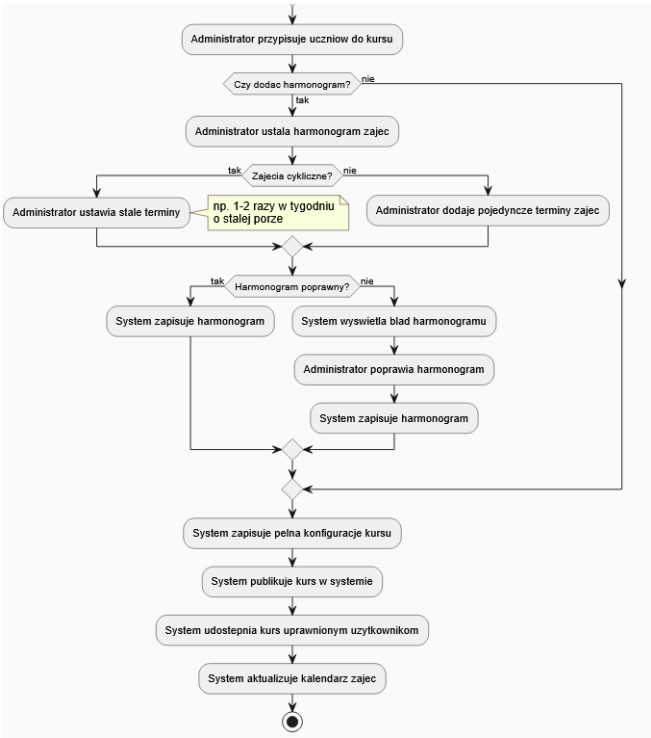
[na poziomie ogólnym i rozszerzonym

**-Diagram przypadków użycia**

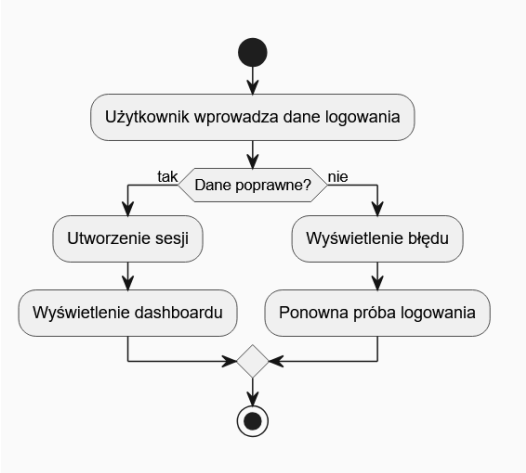


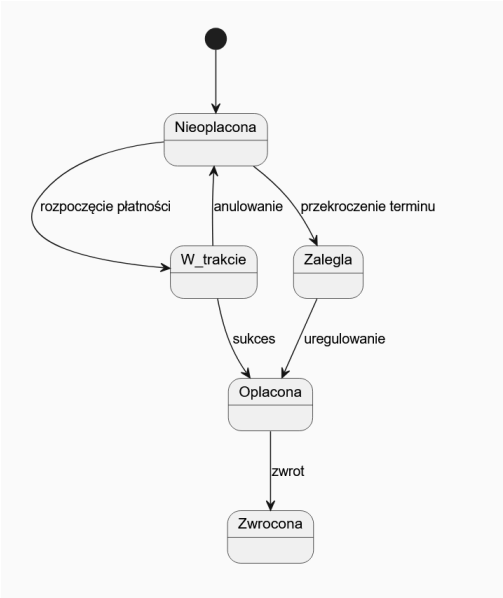
- **Przykładowy diagram aktywności/stanów dla zarządzania kursem przez administratora**



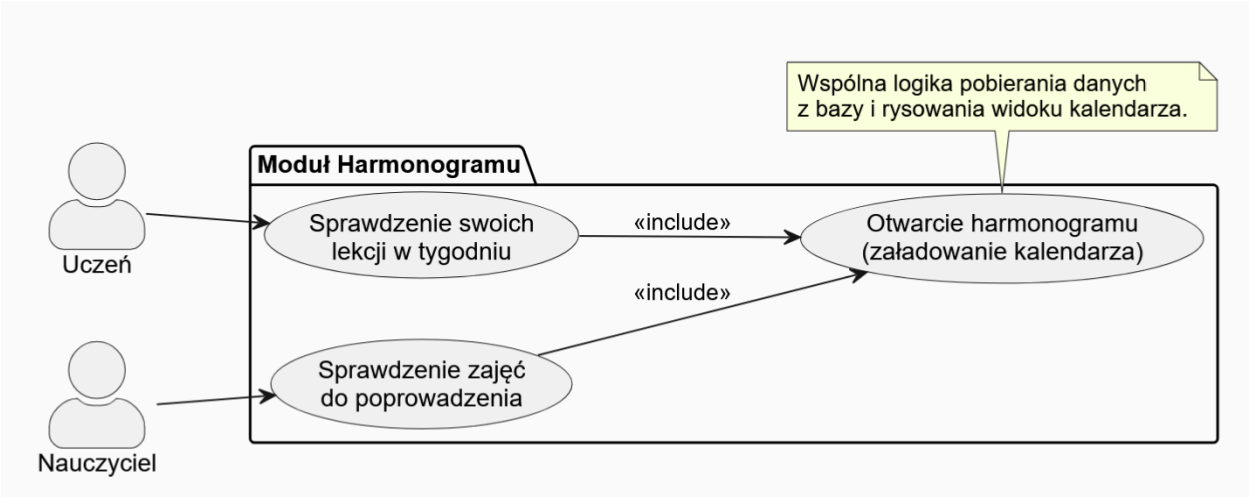



**- Diagramy aktywności/stanów dla skomplikowanych przypadków**

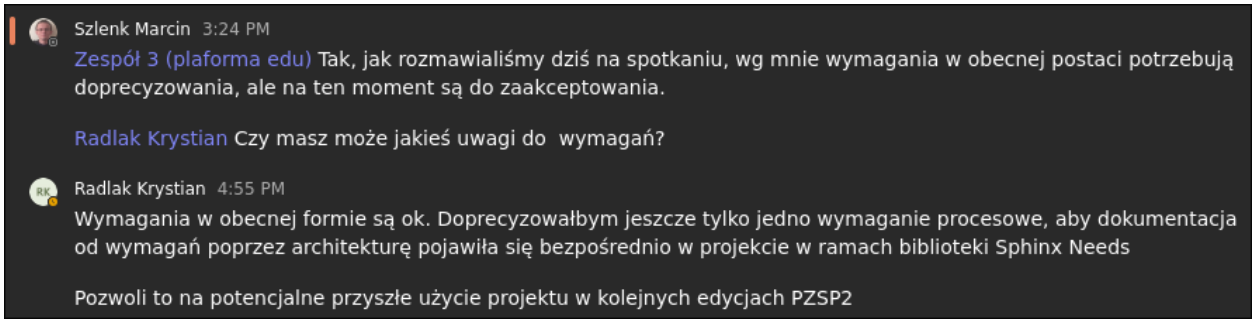



**- Związki pomiędzy przypadkami użycia**



## 3.4 Potwierdzenie zgodności wymagań


Szlenk Marcin 3:24 PM
Zespół 3 (plaforma edu) Tak, jak rozmawialiśmy dziś na spotkaniu, wg mnie wymagania w obecnej postaci potrzebują doprecyzowania, ale na ten moment są do zaakceptowania.

Radlak Krystian Czy masz może jakieś uwagi do wymagań?

Radlak Krystian 4:55 PM
Wymagania w obecnej formie są ok. Doprecyzowałbym jeszcze tylko jedno wymaganie procesowe, aby dokumentacja od wymagań poprzez architekturę pojawiła się bezpośrednio w projekcie w ramach biblioteki Sphinx Needs

Pozwoli to na potencjalne przyszłe użycie projektu w kolejnych edycjach PZSP2

# 4 Definicja architektury

[plan struktury systemu – model komponentów, modułów, serwisów

zastosowane szablony architektoniczne (np. MVC, mikroserwisy, SOA, szyna usług)

interfejsy kluczowych elementów struktury oraz wyjaśnienie połączeń i interakcji pomiędzy nimi

perspektywa logiczna/funkcjonalna oraz perspektywa sprzętowa (tzw. deployment): systemy operacyjne, serwer aplikacyjny, system bazy danych i inne mechanizmy utrwalania danych, system raportowania , system analityczny/BI, mechanizmy zarządzania, mechanizmy bezpieczeństwa

diagramy]

## Dane trwałe

## 4.3 Model logiczny danych

[diagramy ]

### 4.4 Przetwarzanie i przechowywanie danych

[opis planowanego sposobu implementacji: jaka baza danych lub inne oprogramowanie, jak będzie realizowany dostęp do danych (użyte API itp.), czy planowane jest programowanie w b.d. (wyzwalacze, mechanizmy optymalizacji zapytań, itp.)]

## Specyfikacja analityczna i projektowa

Obowiązkowo odnośnik do repozytorium kodu (jedno repozytorium na projekt, jeżeli więcej proszę uzasadnić)

Obowiązkowo określenie metod realizacji: języki programowania, frameworki, środowisko programowania/ uruchamiania/ wdrażania, środowisko ciągłej integracji]

Obowiązkowo Diagram klas lub model pojęciowy struktury informacyjnej: E-R l

Opcjonalnie model struktury systemu (diagram wdrożenia)

Opcjonalnie specyfikacja realizacji przypadków użycia: diagramy sekwencji lub współpracy

Obowiązkowo statystyki: licaba plików, linie kodu, liczba testów jednostkowych

## Projekt standardu interfejsu użytkownika

[wykorzystanie narzędzi do modelowania oraz tworzenia makiet warstwy prezentacyjnej (np. storyboards, wireframes, wireflows, mockups, prototypes etc)]

## Specyfikacja testów

[standardy obsługi błędów i sytuacji wyjątkowych

rodzaje testów, specyfikacja i opis sposobu realizacji poszczególnych rodzajów testów, scenariusze testowe

miary jakości testów]

## *Wirtualizacja/konteneryzacja*

## Bezpieczeństwo

## Podręcznik użytkownika

[instrukcja użycia funkcjonalności systemu]

## Podręcznik administratora

[- instrukcja budowy systemu z kodu źródłowego

- instrukcja instalacji i konfiguracji systemu

- instrukcja aktualizacji oprogramowania

- instrukcja zarządzania użytkownikami i uprawnieniami

- instrukcja tworzenia kopii zapasowych i odtwarzania systemu

- instrukcja zarządzania zasobami systemu]

## Podsumowanie

[Krytyczna analiza osiągniętych wyników, mocne i słabe strony

Możliwe kierunki rozwoju]

## Bibliografia

[Wykaz materiałów źródłowych, opis zgodny ze standardem sporządzania opisów bibliograficznych - https://bg.pw.edu.pl/index.php/przypisy-i-bibliografia]